



Что мы сегодня собираем

Сейчас я продолжаю работать с тем же проектом:

```
hermes-multiagent
```

На прошлом занятии мы уже создали `AGENTS.md` и разобрались с проектным контекстом. Но `AGENTS.md` не подходит для хранения всей кодовой базы. Если проект содержит сотни или тысячи файлов, передавать их целиком в контекст каждого запроса нерационально.

Поэтому сегодня я сделаю отдельный поисковый слой.

Сначала мы разобьём файлы проекта на небольшие chunks. Затем для каждого chunk получим числовой вектор. Эти векторы сохраним в локальном индексе. После этого пользователь сможет задавать вопрос обычным языком, поисковый скрипт найдёт наиболее близкие chunks, а Hermes получит только найденные фрагменты.

В результате запрос вида:

```
Какие части проекта связаны со списанием оплаты заказа через старый платёжный шлюз?
```

не потребует отправлять модели весь репозиторий.

Открываем наш Project в Hermes

Я запускаю Hermes Desktop и открываю уже созданный ранее Project:

```
dependency-lab
```

Рабочая директория должна быть корнем учебного проекта:

```
hermes-multiagent
```

Справа открываю Terminal. Если терминал скрыт, нажимаю:

```
Ctrl + `
```

Проверяю директорию:

```
Get-Location
```

Я хочу увидеть полный путь к открытому проекту.

Если терминал открыт не в корне проекта, выбираю его в File Browser или перехожу в него. Затем сохраняю текущий путь в переменную и проверяю наличие виртуального окружения:

```
$projectRoot = (Get-Location).Path

if (-not (Test-Path -LiteralPath (Join-Path $projectRoot ".venv\Scripts\python.exe") -PathType Leaf)) {
    throw "Откройте корень учебного проекта и повторите проверку."
}
```

После этого проверяю виртуальное окружение:

```
.\.venv\Scripts\python.exe --version
```

Добавляем небольшую кодовую базу для эксперимента

Наш текущий учебный проект слишком маленький для наглядного RAG, поэтому добавим несколько модулей. Они будут находиться внутри того же репозитория, но не будут влиять на предыдущую работу.

Я возвращаюсь в центральный чат Hermes и отправляю:

```
В текущем проекте создай учебную кодовую базу в каталоге demo_codebase.

Создай указанные далее файлы строго с приведённым содержимым.

Другие существующие файлы проекта не изменяй.
```

После этого передаю содержимое файлов.

Файл:

```
demo_codebase/payments/gateway.py
```

```
class LegacyPaymentGateway:
    """Старый клиент платёжного сервиса."""

    def charge_legacy(self, order_id: int, amount: float) -> dict:
        return {
            "order_id": order_id,
            "amount": amount,
            "status": "paid",
            "gateway": "legacy",
        }
```

```
class PaymentGatewayV2:
    """Новый клиент платёжного сервиса."""

    def charge(self, order_id: int, amount: float) -> dict:
        return {
            "order_id": order_id,
            "amount": amount,
            "status": "paid",
            "gateway": "v2",
        }
```

Файл:

demo_codebase/payments/service.py

```
from demo_codebase.payments.gateway import LegacyPaymentGateway

def process_payment(order_id: int, amount: float) -> dict:
    """Выполняет списание оплаты за заказ."""

    gateway = LegacyPaymentGateway()

    return gateway.charge_legacy(
        order_id=order_id,
        amount=amount,
    )
```

Файл:

demo_codebase/orders/checkout.py

```
from demo_codebase.payments.service import process_payment

def checkout(order: dict) -> dict:
    """Завершает оформление заказа и инициирует оплату."""

    payment = process_payment(
        order_id=order["id"],
        amount=order["total"],
    )

    return {
        "order": order,
```

```
"payment": payment,  
}
```

Файл:

```
demo_codebase/notifications/email.py
```

```
def send_order_email(email: str, order_id: int) -> str:  
    """Формирует уведомление о заказе."""  
  
    return f"Уведомление о заказе {order_id} отправлено на {email}"
```

Файл:

```
demo_codebase/auth/session.py
```

```
def create_session(user_id: int) -> dict:  
    """Создаёт пользовательскую сессию."""  
  
    return {  
        "user_id": user_id,  
        "active": True,  
    }
```

После выполнения справа в File Browser раскрываю:

```
demo_codebase
```

Я должен увидеть каталоги `payments`, `orders`, `notifications` и `auth`.

Теперь у нас есть связанные между собой файлы и несколько нерелевантных файлов. Именно это нам необходимо для проверки retrieval.

Создаём каталог RAG

В терминале выполняю:

```
New-Item -ItemType Directory -Force rag
```

Затем:

```
New-Item -ItemType Directory -Force rag\index
```

Проверяю:

```
Get-ChildItem rag
```

Должны существовать:

```
rag  
rag\index
```

Подготавливаем установку библиотеки для embeddings

Для учебного проекта я буду использовать `sentence-transformers`.

Создаю отдельный файл зависимостей:

```
rag-requirements.txt
```

В чат Hermes отправляю:

Создай в корне проекта файл rag-requirements.txt.

Содержимое:

```
sentence-transformers  
numpy
```

Другие файлы не изменяй.

`sentence-transformers` дополнительно загружает PyTorch, а при первом запуске потребуется embedding-модель. Поэтому агент не должен самостоятельно выполнять эту тяжёлую загрузку.

Создаю каталог:

```
New-Item -ItemType Directory -Force scripts
```

Затем создаю файл:

```
scripts\Install-RagDependenciesAndBuildIndex.ps1
```

со следующим содержимым:

```
# Устанавливает зависимости RAG и строит локальный индекс проекта.  
Set-StrictMode -Version Latest
```

```

$ErrorActionPreference = "Stop"

$projectRoot = Split-Path -Parent $PSScriptRoot
$pythonPath = Join-Path $projectRoot ".venv\Scripts\python.exe"
$requirementsPath = Join-Path $projectRoot "rag-requirements.txt"
$buildIndexPath = Join-Path $projectRoot "rag\build_index.py"

if (-not (Test-Path -LiteralPath $pythonPath -PathType Leaf)) {
    throw "Не найден Python виртуального окружения: $pythonPath"
}

if (-not (Test-Path -LiteralPath $requirementsPath -PathType Leaf)) {
    throw "Не найден файл зависимостей RAG: $requirementsPath"
}

if (-not (Test-Path -LiteralPath $buildIndexPath -PathType Leaf)) {
    throw "Не найден индексатор RAG: $buildIndexPath"
}

Write-Warning "Будут загружены sentence-transformers, PyTorch и embedding-модель. Это может занять продолжительное время и потребовать значительный объём диска."

& $pythonPath -m pip install -r $requirementsPath
if ($LASTEXITCODE -ne 0) {
    exit $LASTEXITCODE
}

& $pythonPath $buildIndexPath
if ($LASTEXITCODE -ne 0) {
    exit $LASTEXITCODE
}

```

Пока скрипт не запускаю: сначала необходимо создать `rag\build_index.py`. После этого пользователь самостоятельно запустит подготовку командой:

```
powershell -ExecutionPolicy Bypass -File .\scripts\Install-RagDependenciesAndBuildIndex.ps1
```

Скрипт заранее предупреждает о тяжёлой загрузке, устанавливает зависимости и затем строит индекс. Агент не запускает его без явного указания пользователя.

Для embeddings я использую модель:

```
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
```

Она подходит для нашего учебного варианта, потому что мы будем задавать вопросы на русском языке, а имена классов и функций в проекте преимущественно английские.

При первом запуске модель загрузится локально. На последующих запусках будет использоваться кэш.

Создаём индексатор

Теперь создадим программу, которая прочитает кодовую базу, разобьёт её на chunks и построит векторный индекс.

В центральном чате Hermes отправляю:

Создай файл:

rag\build_index.py

строго со следующим содержимым.

Код:

```
from __future__ import annotations

import json
from pathlib import Path

import numpy as np
from sentence_transformers import SentenceTransformer

PROJECT_ROOT = Path(__file__).resolve().parents[1]

INDEX_DIR = PROJECT_ROOT / "rag" / "index"

MODEL_NAME = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"

ALLOWED_EXTENSIONS = {
    ".py",
    ".md",
    ".txt",
    ".json",
    ".yaml",
    ".yml",
    ".toml",
}

EXCLUDED_DIRECTORIES = {
    ".git",
    ".venv",
    ".hermes",
    "__pycache__",
    "node_modules",
    "models",
}
```

```
CHUNK_SIZE_LINES = 60
CHUNK_OVERLAP_LINES = 15

def should_index(path: Path) -> bool:
    """Определяет, нужно ли добавлять файл в индекс."""

    if not path.is_file():
        return False

    if path.suffix.lower() not in ALLOWED_EXTENSIONS:
        return False

    relative_path = path.relative_to(PROJECT_ROOT)

    if any(part in EXCLUDED_DIRECTORIES for part in relative_path.parts):
        return False

    if (
        len(relative_path.parts) >= 2
        and relative_path.parts[0] == "rag"
        and relative_path.parts[1] == "index"
    ):
        return False

    return True

def create_chunks(path: Path) -> list[dict]:
    """Разбивает текстовый файл на перекрывающиеся части."""

    text = path.read_text(
        encoding="utf-8",
        errors="ignore",
    )

    lines = text.splitlines()

    if not lines:
        return []

    chunks = []

    step = CHUNK_SIZE_LINES - CHUNK_OVERLAP_LINES

    start = 0

    while start < len(lines):
        end = min(
            start + CHUNK_SIZE_LINES,
            len(lines),

```

```
)

content = "\n".join(lines[start:end]).strip()

if content:
    relative_path = str(path.relative_to(PROJECT_ROOT))

    chunks.append(
        {
            "path": relative_path,
            "start_line": start + 1,
            "end_line": end,
            "content": content,
        }
    )

if end >= len(lines):
    break

start += step

return chunks

def main() -> None:
    """Строит локальный векторный индекс проекта."""

    INDEX_DIR.mkdir(
        parents=True,
        exist_ok=True,
    )

    files = sorted(path for path in PROJECT_ROOT.rglob("*") if should_index(path))

    chunks = []

    for path in files:
        chunks.extend(create_chunks(path))

    if not chunks:
        raise RuntimeError("Не найдено ни одного фрагмента для индексации.")

    texts = []

    for chunk in chunks:
        texts.append(
            "Путь файла: "
            f"{chunk['path']}\n"
            "Строки: "
            f"{chunk['start_line']}-"
            f"{chunk['end_line']}\n\n"
```

```
        f"{chunk['content']}")
    )

print(f"Найдено файлов: {len(files)}")

print(f"Создано фрагментов: {len(chunks)}")

print(f"Загружается embedding-модель: {MODEL_NAME}")

model = SentenceTransformer(MODEL_NAME)

vectors = model.encode(
    texts,
    batch_size=16,
    show_progress_bar=True,
    normalize_embeddings=True,
    convert_to_numpy=True,
)

vectors = np.asarray(
    vectors,
    dtype=np.float32,
)

np.save(
    INDEX_DIR / "vectors.npy",
    vectors,
)

with (INDEX_DIR / "chunks.json").open(
    "w",
    encoding="utf-8",
) as file:
    json.dump(
        chunks,
        file,
        ensure_ascii=False,
        indent=2,
    )

metadata = {
    "model": MODEL_NAME,
    "files_count": len(files),
    "chunks_count": len(chunks),
    "vector_dimension": int(vectors.shape[1]),
    "chunk_size_lines": CHUNK_SIZE_LINES,
    "chunk_overlap_lines": (CHUNK_OVERLAP_LINES),
}

with (INDEX_DIR / "meta.json").open(
    "w",
```

```
        encoding="utf-8",
    ) as file:
        json.dump(
            metadata,
            file,
            ensure_ascii=False,
            indent=2,
        )

    print()
    print("Индекс успешно создан.")
    print(f"Каталог индекса: {INDEX_DIR}")

if __name__ == "__main__":
    main()
```

Что именно делает индексатор

Перед запуском я коротко разбираю со студентами саму механику.

Программа проходит по проекту и выбирает текстовые файлы. `.venv`, `.git`, локальные модели и служебный каталог `.hermes` специально исключены.

Каждый файл разбивается на участки по 60 строк.

При этом между соседними фрагментами сохраняется перекрытие в 15 строк. Это нужно для того, чтобы смысл не обрывался строго на границе chunk.

Для каждого полученного фрагмента embedding-модель создаёт числовой вектор.

В результате мы сохраняем два основных массива данных.

В `chunks.json` находится исходный текст и информация о файлах.

В `vectors.npy` находятся соответствующие векторы.

Это и есть наш простой локальный векторный индекс.

Запускаем индексацию

Если это первый запуск, пользователь выполняет подготовленный скрипт, который установит зависимости и построит индекс:

```
powershell -ExecutionPolicy Bypass -File .\scripts\Install-RagDependenciesAndBuildIndex.ps1
```

Если зависимости и модель уже установлены, для повторной индексации достаточно выполнить:

```
.\.venv\Scripts\python.exe .\rag\build_index.py
```

При первом запуске модель может сначала загрузиться.

После завершения я ожидаю сообщения:

```
Индекс успешно создан.
```

Теперь в File Browser раскрываю:

```
rag  
index
```

Там должны появиться:

```
chunks.json  
meta.json  
vectors.npy
```

Открываю `meta.json`.

Например, там будет информация о количестве файлов, числе chunks и размерности векторов.

Открываю `chunks.json`.

Здесь я уже могу визуально показать студентам, что индекс хранит не целые файлы, а отдельные части.

`vectors.npy` открывать как обычный текст не нужно. Это бинарный NumPy-массив.

Создаём поисковый скрипт

Теперь нам нужен retrieval.

В Hermes отправляю:

```
Создай файл:  
  
rag\search_index.py  
  
строго со следующим содержимым.
```

Код:

```
from __future__ import annotations
```

```
import argparse
import json
from pathlib import Path

import numpy as np
from sentence_transformers import SentenceTransformer

PROJECT_ROOT = Path(__file__).resolve().parents[1]

INDEX_DIR = PROJECT_ROOT / "rag" / "index"

def load_index() -> tuple[
    list[dict],
    np.ndarray,
    dict,
]:
    """Загружает метаданные, chunks и векторы."""

    chunks_path = INDEX_DIR / "chunks.json"

    vectors_path = INDEX_DIR / "vectors.npy"

    meta_path = INDEX_DIR / "meta.json"

    if not chunks_path.exists():
        raise FileNotFoundError("Индекс не найден. Сначала запустите build_index.py.")

    with chunks_path.open(
        "r",
        encoding="utf-8",
    ) as file:
        chunks = json.load(file)

    vectors = np.load(vectors_path)

    with meta_path.open(
        "r",
        encoding="utf-8",
    ) as file:
        metadata = json.load(file)

    return chunks, vectors, metadata

def search(
    query: str,
    top_k: int,
) -> dict:
```

```
"""Ищет наиболее близкие chunks."""

chunks, vectors, metadata = load_index()

model = SentenceTransformer(metadata["model"])

query_vector = model.encode(
    [query],
    normalize_embeddings=True,
    convert_to_numpy=True,
)[0]

query_vector = np.asarray(
    query_vector,
    dtype=np.float32,
)

scores = vectors @ query_vector

top_k = max(
    1,
    min(
        top_k,
        len(chunks),
        8,
    ),
)

indexes = np.argsort(scores)[::-1][:top_k]

results = []

for index in indexes:
    chunk = chunks[int(index)]

    results.append(
        {
            "score": round(
                float(scores[index]),
                4,
            ),
            "path": chunk["path"],
            "start_line": (chunk["start_line"]),
            "end_line": (chunk["end_line"]),
            "content": (chunk["content"][:4000]),
        }
    )

return {
    "query": query,
    "results": results,
```

```
}

def main() -> None:
    """CLI для проверки векторного поиска."""

    parser = argparse.ArgumentParser()

    parser.add_argument(
        "--query",
        required=True,
    )

    parser.add_argument(
        "--top-k",
        type=int,
        default=4,
    )

    arguments = parser.parse_args()

    result = search(
        query=arguments.query,
        top_k=arguments.top_k,
    )

    print(
        json.dumps(
            result,
            ensure_ascii=False,
            indent=2,
        )
    )

if __name__ == "__main__":
    main()
```

Проверяем retrieval без Hermes

Очень важно сначала проверить поисковый слой отдельно.

Если сразу подключить всё к агенту, а результат окажется неправильным, будет трудно понять, проблема в индексе, embedding-модели, plugin или самом агенте.

В Terminal выполняю:

```
.\.venv\Scripts\python.exe .\rag\search_index.py --query "Где в проекте происходит списание"
```

```
оплаты заказа через старый платёжный шлюз?" --top-k 4
```

Теперь смотрю результаты.

В верхней части выдачи я ожидаю увидеть прежде всего:

```
demo_codebase\payments\service.py
```

и:

```
demo_codebase\payments\gateway.py
```

Также вполне логично увидеть:

```
demo_codebase\orders\checkout.py
```

Файлы уведомлений или пользовательской сессии должны иметь меньшую релевантность.

Это очень важная точка практики.

ИИ ещё вообще не участвовал в ответе.

Мы проверили только retrieval.

Проверяем другой запрос

Теперь выполняю:

```
.\.venv\Scripts\python.exe .\rag\search_index.py --query "Где создаётся пользовательская сес-  
сия?" --top-k 3
```

Здесь среди первых результатов должен появиться:

```
demo_codebase\auth\session.py
```

Теперь мы знаем, что индекс работает.

Создаём plugin для Hermes

Следующий этап — дать Hermes возможность пользоваться нашим поиском.

Для проекта создаём локальный plugin:

```
.hermes\plugins\code-rag
```

В Terminal:

```
New-Item -ItemType Directory -Force .hermes\plugins\code-rag
```

Обратите внимание: project-local plugins в Hermes по умолчанию отключены из соображений безопасности. Для доверенного проекта их необходимо явно разрешить через

```
HERMES_ENABLE_PROJECT_PLUGINS=true
```

Пока Hermes перезапускать не нужно. Сначала создадим сам plugin.

Создаём plugin.yaml

В центральном чате отправляю:

Создай файл:

```
.hermes\plugins\code-rag\plugin.yaml
```

строго со следующим содержимым.

```
name: code-rag
version: "1.0.0"
description: Локальный RAG-поиск по кодовой базе проекта
provides_tools:
  - codebase_search
python_runtime: external
```

Манифест сообщает Hermes, что plugin называется `code-rag` и предоставляет инструмент `codebase_search`. Такая схема соответствует текущему plugin API Hermes.

Создаём сам инструмент

Теперь создаём:

```
.hermes\plugins\code-rag\__init__.py
```

В чат Hermes отправляю:

Создай файл:

```
.hermes\plugins\code-rag\__init__.py
```

строго со следующим содержимым.

Код:

```
from __future__ import annotations

import json
import os
import subprocess
from pathlib import Path

PLUGIN_DIR = Path(__file__).resolve().parent

PROJECT_ROOT = PLUGIN_DIR.parents[2]

PYTHON_EXECUTABLE = PROJECT_ROOT / ".venv" / "Scripts" / "python.exe"

SEARCH_SCRIPT = PROJECT_ROOT / "rag" / "search_index.py"

def register(ctx):
    """Регистрирует инструмент RAG-поиска."""

    schema = {
        "name": "codebase_search",
        "description": (
            "Выполняет семантический поиск "
            "по локальному векторному индексу "
            "кодовой базы текущего проекта. "
            "Используй этот инструмент перед "
            "ответом на вопросы о структуре "
            "проекта, бизнес-логике, зависимостях, "
            "местах использования API и файлах, "
            "которые могут быть связаны с изменением."
        ),
        "parameters": {
            "type": "object",
            "properties": {
                "query": {
                    "type": "string",
                    "description": ("Поисковый запрос естественным языком."),
                },
                "top_k": {
                    "type": "integer",
                    "description": ("Количество возвращаемых фрагментов от 1 до 8."),
                    "minimum": 1,
                    "maximum": 8,
                },
            },
        },
    }
```

```
        },
    },
    "required": [
        "query",
    ],
},
}

def handle_search(
    params,
    **kwargs,
):
    """Запускает поиск в отдельном Python-процессе."""

    del kwargs

    query = str(
        params.get(
            "query",
            "",
        )
    ).strip()

    if not query:
        return json.dumps(
            {"error": ("Поисковый запрос пуст.")},
            ensure_ascii=False,
        )

    raw_top_k = params.get(
        "top_k",
        4,
    )

    if isinstance(raw_top_k, bool) or isinstance(raw_top_k, float):
        return json.dumps(
            {"error": ("Параметр top_k должен быть целым числом от 1 до 8.")},
            ensure_ascii=False,
        )

    try:
        top_k = int(raw_top_k)
    except (TypeError, ValueError):
        return json.dumps(
            {"error": ("Параметр top_k должен быть целым числом от 1 до 8.")},
            ensure_ascii=False,
        )

    if not 1 <= top_k <= 8:
        return json.dumps(
            {"error": ("Параметр top_k должен быть в диапазоне от 1 до 8.")},
```

```
        ensure_ascii=False,
    )

    if not PYTHON_EXECUTABLE.exists():
        return json.dumps(
            {
                "error": (
                    f"Не найден Python виртуального окружения: {PYTHON_EXECUTABLE}"
                )
            },
            ensure_ascii=False,
        )

    if not SEARCH_SCRIPT.exists():
        return json.dumps(
            {"error": (f"Не найден скрипт поиска: {SEARCH_SCRIPT}")},
            ensure_ascii=False,
        )

    environment = dict(os.environ)

    environment["PYTHONUTF8"] = "1"

    command = [
        str(PYTHON_EXECUTABLE),
        str(SEARCH_SCRIPT),
        "--query",
        query,
        "--top-k",
        str(top_k),
    ]

    try:
        process = subprocess.run(
            command,
            cwd=str(PROJECT_ROOT),
            capture_output=True,
            text=True,
            encoding="utf-8",
            errors="replace",
            timeout=120,
            env=environment,
        )

    except subprocess.TimeoutExpired:
        return json.dumps(
            {"error": ("Поиск превысил лимит времени выполнения.")},
            ensure_ascii=False,
        )

    if process.returncode != 0:
```

```
        return json.dumps(
            {"error": (process.stderr.strip() or "Ошибка поиска.")},
            ensure_ascii=False,
        )

    return process.stdout.strip() or json.dumps(
        {"error": ("Поиск не вернул данные.")},
        ensure_ascii=False,
    )

ctx.register_tool(
    name="codebase_search",
    toolset="code_rag",
    schema=schema,
    handler=handle_search,
)
```

Здесь есть важный технический момент.

Сам Hermes plugin не загружает `sentence-transformers`. Он вызывает Python из нашего `.venv`, где библиотека уже установлена.

Это позволяет не смешивать зависимости самого Hermes и зависимости учебного RAG-проекта.

Проверяем plugin до запуска Hermes

Теперь я обязательно запускаю Plugin Doctor.

В Terminal убеждаюсь, что открыт корень учебного проекта:

```
$projectRoot = (Get-Location).Path

if (-not (Test-Path -LiteralPath (Join-Path $projectRoot ".hermes\plugins\code-rag") -PathType
Container)) {
    throw "Откройте корень учебного проекта и повторите проверку."
}
```

Выполняю:

```
hermes plugins doctor .\hermes\plugins\code-rag --ci
```

`hermes plugins doctor` использует настоящий parser, loader и registry Hermes и проверяет, может ли plugin зарегистрироваться корректно. Это значительно полезнее, чем просто проверять Python-файл на синтаксис.

Если Doctor сообщает ошибку, дальше не продолжаю.

Сначала исправляю plugin.

Разрешаем project-local plugins

Теперь нужно перезапустить Hermes Desktop.

Закрываю текущее окно Hermes Desktop.

Открываю обычный PowerShell Windows в корне учебного проекта. Сохраняю текущий путь и проверяю его:

```
$projectRoot = (Get-Location).Path

if (-not (Test-Path -LiteralPath (Join-Path $projectRoot ".hermes\plugins\code-rag") -PathType Container)) {
    throw "Откройте корень учебного проекта и повторите проверку."
}
```

Включаю project-local plugins для этого процесса:

```
$env:HERMES_ENABLE_PROJECT_PLUGINS="true"
```

Теперь проверяю обнаружение:

```
hermes plugins list
```

Я должен увидеть:

```
code-rag
```

Если plugin найден, включаю его:

```
hermes plugins enable code-rag
```

Hermes специально требует явного включения general plugins; обнаруженный plugin сам по себе ещё не означает, что его код будет выполняться.

Теперь запускаю Desktop из этого же PowerShell:

```
hermes desktop --cwd $projectRoot
```

Ключевой момент заключается в том, что переменная:

```
HERMES_ENABLE_PROJECT_PLUGINS
```

действует для запущенного из этого PowerShell процесса.

Если просто закрыть PowerShell, а затем запустить Hermes обычным ярлыком, project-local plugin может снова не загрузиться.

Проверяем инструмент в Hermes

После запуска открываю новый чат.

В поле сообщения ввожу:

```
/tools list
```

Hermes поддерживает `/tools list`, `/tools enable` и `/tools disable` для управления инструментами текущей сессии.

Ищу:

```
codebase_search
```

Если инструмент отображается, но отключён, ввожу:

```
/tools enable codebase_search
```

После изменения набора инструментов Hermes может начать новую сессию, потому что изменяется tool schema.

Затем ещё раз:

```
/tools list
```

`codebase_search` должен быть доступен.

Проверяем инструмент непосредственно через агента

Теперь я ещё не прошу анализировать архитектуру.

Сначала даю максимально явную команду:

Используй инструмент `codebase_search`.

Найди в кодовой базе информацию по запросу:

"Где происходит списание оплаты заказа через старый платёжный шлюз?"

Верни названия найденных файлов и диапазоны строк.

Теперь Hermes должен вызвать:

```
codebase_search
```

В интерфейсе я раскрываю карточку вызова инструмента, если она отображается свёрнутой.

Я хочу увидеть, что инструмент получил строку запроса и вернул несколько chunks.

Теперь происходит самый важный момент всей практики.

`search_index.py` нашёл фрагменты.

Plugin вернул их Hermes.

Ответ инструмента становится частью текущего tool-calling контекста модели.

Только после этого модель формирует человеческий ответ.

Это и есть retrieval-augmented generation в нашей реализации.

Добавляем правило использования RAG в AGENTS.md

Пока мы заставили модель вызвать поиск напрямую.

Но нам нужно, чтобы агент сам понимал, когда использовать RAG.

Открываю корневой:

```
AGENTS.md
```

В центральном чате отправляю:

В конец корневого AGENTS.md добавь следующий раздел.
Другой текст файла не изменяй.

Добавляю:

```
## Codebase RAG
```

При вопросах о структуре проекта, связях между модулями, бизнес-логике, зависимостях, местах использования API или потенциально затрагиваемых файлах сначала используй инструмент `codebase_search`.

Не делай вывод о всей кодовой базе только по одному найденному фрагменту.

Если первые результаты недостаточны, выполни повторный поиск с уточнённым запросом.

В итоговом ответе указывай пути файлов, на основании которых сделан вывод.

После изменения `AGENTS.md` я создаю новую сессию, чтобы проектный контекст был загружен чисто.

Hermes загружает `AGENTS.md` в системный проектный контекст и поддерживает его иерархию внутри репозитория.

Проверяем уже автоматический выбор retrieval

В новом чате больше не пишу название инструмента.

Просто задаю:

Какие части проекта участвуют в списании оплаты заказа через старый платёжный шлюз и какие файлы потенциально потребуется изменить при переходе на новый платёжный клиент?

Теперь ожидаю следующую последовательность.

Сначала Hermes сам вызывает `codebase_search`.

В результатах retrieval появляются `payments/gateway.py`, `payments/service.py` и `orders/checkout.py`.

После этого агент анализирует найденные chunks и формирует ответ.

Вот здесь мы уже получили полноценный agentic RAG: пользователь сформулировал обычную техническую задачу, а выбор retrieval-инструмента выполнил агент.

Проверяем уточняющий retrieval

Теперь задаю более сложный вопрос:

Определи путь выполнения операции оплаты от оформления заказа до непосредственного вызова старого платёжного API.

Первого retrieval может оказаться достаточно.

Но если контекста мало, агент в соответствии с `AGENTS.md` может выполнить ещё один запрос, например более узкий поиск по `process_payment` или старому gateway.

Это важное отличие RAG от одноразового поиска.

Retrieval можно выполнять итеративно.

Агент получает первый контекст, понимает, какой информации не хватает, выполняет новый retrieval и расширяет рабочий контекст только нужными данными.

Смотрим, сколько контекста мы реально используем

В Hermes ввожу:

```
/context
```

или для более подробного просмотра:

```
/context all
```

Hermes показывает распределение текущего контекстного окна между системными инструкциями, инструментами, памятью, разговором и свободным пространством.

Теперь я могу показать студентам принципиальную разницу.

Мы не отправили в запрос всю кодовую базу.

В контекст попали системные инструкции, `AGENTS.md`, описание инструмента и несколько chunks, которые retrieval признал релевантными.

Именно таким образом RAG помогает управлять контекстным окном.

Сравниваем RAG и отсутствие RAG

Теперь проведём простой эксперимент.

Отключаю инструмент:

```
/tools disable codebase_search
```

Начинаю новую сессию и задаю:

```
Какие части проекта участвуют в списании оплаты заказа через старый платёжный шлюз?
```

Hermes всё ещё может попытаться использовать обычные файловые инструменты и исследовать проект самостоятельно.

Но теперь специализированного семантического retrieval у него нет.

Возвращаю инструмент:

```
/tools enable codebase_search
```

И повторяю запрос в новой сессии.

Здесь студентам нужно увидеть разницу не обязательно в финальной формулировке ответа, а в способе получения контекста.

Без RAG агент сам исследует файловую систему.

С RAG он сначала получает ограниченный набор наиболее релевантных chunks.

Проверяем проблему устаревшего индекса

Теперь покажу ещё одну важную проблему RAG-систем.

Открываю:

```
demo_codebase\payments\service.py
```

и добавляю новую функцию через Hermes:

```
def get_payment_provider_name() -> str:  
    return "legacy-payment-provider"
```

Файл изменился.

Но векторный индекс мы пока не перестраивали.

Запускаю:

```
.\.venv\Scripts\python.exe .\rag\search_index.py --query "Где определяется название платёжного  
провайдера?" --top-k 4
```

Новая информация может отсутствовать, потому что `chunks.json` и `vectors.npy` были созданы до изменения файла.

Это принципиальный момент.

RAG не читает файлы в реальном времени автоматически.

Он читает индекс.

Если индекс устарел, агент получает устаревший контекст.

Переиндексируем проект

Для обновления выполняю:

```
.\.venv\Scripts\python.exe .\rag\build_index.py
```

Старые:

```
vectors.npy  
chunks.json  
meta.json
```

перезаписываются новой версией.

Теперь снова выполняю:

```
.\.venv\Scripts\python.exe .\rag\search_index.py --query "Где определяется название платёжного провайдера?" --top-k 4
```

Теперь новый фрагмент уже должен участвовать в retrieval.

Таким образом, мы отдельно увидели жизненный цикл индекса:

код изменился;

индекс стал устаревшим;

мы выполнили переиндексацию;

поиск снова соответствует текущему состоянию проекта.

Проверяем семантический поиск

Теперь специально зададим запрос, в котором нет точного имени функции:

```
.\.venv\Scripts\python.exe .\rag\search_index.py --query "Как приложение получает деньги с клиента при оформлении покупки?" --top-k 4
```

В самом коде может нигде не существовать точной фразы:

```
получает деньги с клиента
```

Но embedding-поиск должен попытаться найти смыслово близкие chunks, связанные с оплатой.

Именно здесь векторный retrieval отличается от простого поиска `grep`.

`grep` прекрасно работает, когда я знаю конкретное имя:

```
charge_legacy
```

RAG полезнее, когда пользователь формулирует вопрос на уровне смысла.

Когда использовать обычный поиск, а когда RAG

Я отдельно объясняю студентам, что RAG не заменяет обычные инструменты поиска.

Если я точно знаю имя:

```
LegacyPaymentGateway
```

то обычный поиск по строке может быть быстрее и точнее.

Если же вопрос звучит:

```
Какие части проекта отвечают за оплату заказа?
```

семантический поиск становится значительно полезнее.

В реальной агентной системе эти два подхода обычно дополняют друг друга.

Что происходит внутри нашего RAG по шагам

Когда пользователь задаёт Hermes вопрос о кодовой базе, агент сначала видит правило из `AGENTS.md`, которое требует использовать `codebase_search`.

Hermes вызывает наш plugin-tool.

Plugin запускает:

```
rag/search_index.py
```

Поисковый запрос превращается в embedding.

Этот вектор сравнивается со всеми векторами chunks через скалярное произведение нормализованных векторов, что в нашем случае эквивалентно cosine similarity.

Затем выбираются chunks с наибольшим similarity score.

Их текст возвращается Hermes как результат tool call.

Модель получает найденный материал в текущем контексте и уже после этого формирует ответ.

Именно поэтому здесь есть обе части термина RAG.

`Retrieval` — поиск релевантных chunks.

`Augmented Generation` — генерация ответа с добавленным найденным контекстом.

Почему мы не положили всю кодовую базу в AGENTS.md

Теперь хорошо видно различие между предыдущей и сегодняшней практикой.

`AGENTS.md` содержит устойчивые правила работы с проектом.

Он сообщает агенту, как вести себя и когда использовать retrieval.

Но исходный код туда не копируется.

Сам код находится в репозитории и в нашем векторном индексе.

Когда возникает конкретный вопрос, система извлекает только подходящие части.

Hermes загружает project context files в системную часть контекста, а динамические результаты инструментов появляются только в процессе выполнения конкретного запроса.

Почему для занятия мы используем tool-based RAG

В Hermes существует и более автоматический вариант. Plugin может зарегистрировать `pre_llm_call` hook и перед каждым обращением к модели автоматически добавлять найденный контекст в текущий user turn. Официальная документация прямо называет этот механизм подходящим для RAG integrations.

Но для первого занятия я намеренно не использую автоматическую инъекцию.

Мне важно, чтобы студенты видели отдельный вызов:

```
codebase_search
```

Тогда хорошо заметна архитектура RAG.

Сначала retrieval.

Затем найденный материал.

Затем генерация ответа.

Если спрятать retrieval внутрь автоматического hook уже на первом занятии, значительная часть процесса станет невидимой.

Проверяем итоговую структуру проекта

В Terminal выполняю:

```
Get-ChildItem -Recurse rag
```

Затем:

```
Get-ChildItem -Recurse .hermes\plugins\code-rag
```

У нас должна получиться примерно такая структура:

```
hermes-multiagent
  AGENTS.md

  demo_codebase
    auth
```

```
notifications
orders
payments

rag
  build_index.py
  search_index.py
  index
    chunks.json
    meta.json
    vectors.npy

.hermes
  plugins
    code-rag
      plugin.yaml
      __init__.py

scripts
  Install-RagDependenciesAndBuildIndex.ps1

rag-requirements.txt
```

Финальная проверка всей системы

Теперь я создаю новую сессию Hermes.

Проверяю:

```
/context
```

Затем:

```
/tools list
```

Убеждаюсь, что `AGENTS.md` загружен, а `codebase_search` доступен.

После этого задаю финальный запрос:

Проанализируй кодовую базу проекта.

Найди, какие компоненты участвуют в обработке оплаты заказа через устаревший платёжный клиент.

Определи, какие файлы потенциально потребуется изменить для перехода на новый клиент.

Не просматривай весь проект без необходимости. Используй релевантный контекст из кодовой базы и укажи файлы, на основании которых сделан вывод.

Я ожидаю, что Hermes сначала обратится к `codebase_search`, получит несколько chunks и только после этого сформирует ответ.

Если это произошло, наша RAG-система работает.

Что мы получили в результате

Теперь у нас есть уже не просто AI-агент, который умеет читать файлы.

У нас появился отдельный слой знаний о кодовой базе.

Исходные файлы разбиваются на chunks.

Chunks преобразуются в embeddings.

Embeddings сохраняются в локальном векторном индексе.

По пользовательскому запросу выполняется semantic retrieval.

Hermes получает несколько наиболее релевантных chunks как рабочий контекст.

Модель формирует ответ уже с опорой на найденные материалы.

При этом мы не пытаемся держать весь проект в контекстном окне.

Именно в этом заключается основная идея RAG для кодовой базы: не передавать модели всё, что мы знаем о проекте, а находить и передавать то, что необходимо для конкретной задачи.
